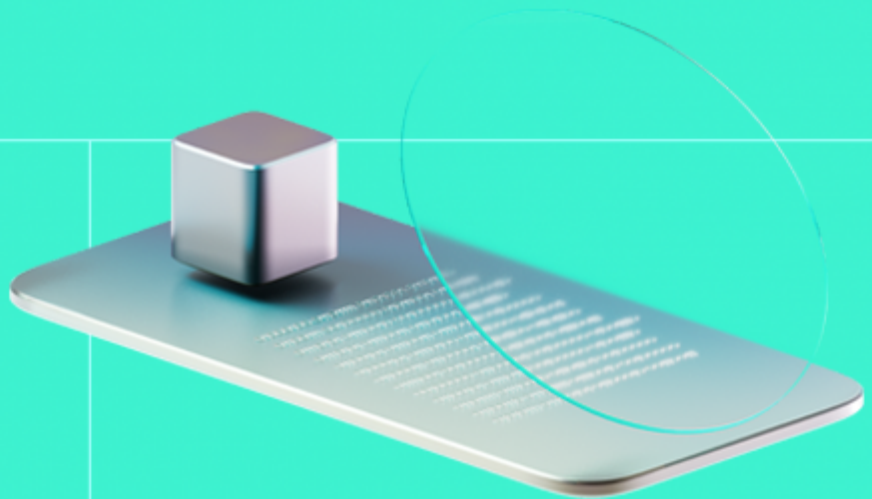




# Smart Contract Code Review And Security Analysis Report

**Customer:** SyFu

**Date:** 19/03/2026



We express our gratitude to the SyFu team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

SyFu is a Web3 wallet with GameFi (Game Finance) features that convert payment data into digital assets.

## Document

|             |                                                                                                                         |
|-------------|-------------------------------------------------------------------------------------------------------------------------|
| Name        | Smart Contract Code Review and Security Analysis Report for SyFu                                                        |
| Audited By  | Khrystyna Tkachuk                                                                                                       |
| Approved By | Kornel Światłowski                                                                                                      |
| Website     | <a href="https://syfu.io/">https://syfu.io/</a>                                                                         |
| Changelog   | 10/03/2026 - Preliminary Report                                                                                         |
|             | 19/03/2026 - Final Report                                                                                               |
| Platform    | BSC                                                                                                                     |
| Language    | Solidity                                                                                                                |
| Tags        | Staking, Upgradable                                                                                                     |
| Methodology | <a href="https://docs.hacken.io/methodologies/smart-contracts">https://docs.hacken.io/methodologies/smart-contracts</a> |

## Review Scope

|                |                                                                                                         |
|----------------|---------------------------------------------------------------------------------------------------------|
| Repository     | <a href="https://github.com/SyFu-Project/syfu_staking">https://github.com/SyFu-Project/syfu_staking</a> |
| Initial Commit | ed75700                                                                                                 |
| Final Commit   | 8ca0aa5                                                                                                 |

# Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

|                |          |          |           |
|----------------|----------|----------|-----------|
| 4              | 4        | 0        | 0         |
| Total Findings | Resolved | Accepted | Mitigated |

## Findings by Severity

| Severity | Count |
|----------|-------|
| Critical | 0     |
| High     | 0     |
| Medium   | 0     |
| Low      | 0     |

| Vulnerability                                                                           | Severity | Status |
|-----------------------------------------------------------------------------------------|----------|--------|
| <a href="#">F-2026-15337</a> - Floating Pragma                                          | Info     | Fixed  |
| <a href="#">F-2026-15341</a> - Lack of Event Emitting for Emergency Withdraw Operations | Info     | Fixed  |
| <a href="#">F-2026-15342</a> - Redundant Imports                                        | Info     | Fixed  |
| <a href="#">F-2026-15344</a> - Inconsistent Granular Role Separation                    | Info     | Fixed  |

## Documentation quality

- Functional requirements are detailed.
  - Project overview is detailed
  - All roles in the system are described.
  - Use cases are described and detailed.
  - All futures are described
- Technical description is detailed.
  - Run instructions are provided.
  - Technical specification is provided.
  - NatSpec is sufficient.

## Code quality

- The project extensively utilizes OpenZeppelin contracts.
- The development environment is configured.

## Test coverage

Code coverage of the project is **80.8%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is partially covered.

# Table of Contents

|                                         |           |
|-----------------------------------------|-----------|
| <b>System Overview</b>                  | <b>6</b>  |
| Privileged Roles                        | 6         |
| <b>Potential Risks</b>                  | <b>7</b>  |
| <b>Findings</b>                         | <b>8</b>  |
| Vulnerability Details                   | 8         |
| Disclaimers                             | 16        |
| <b>Appendix 1. Definitions</b>          | <b>17</b> |
| Severities                              | 17        |
| Potential Risks                         | 17        |
| <b>Appendix 2. Scope</b>                | <b>18</b> |
| <b>Appendix 3. Additional Valuables</b> | <b>19</b> |

## System Overview

`SyFuStaking.sol` is a UUPS-upgradeable vault on BNB Smart Chain for holding EVT tokens under predefined pool-specific lock periods. The contract is responsible only for custody of locked tokens, enforcement of lock durations, and accounting of stake, pool, and global balances. All broader business logic, including reward calculation, pool configuration strategy, eligibility checks, CAP enforcement, snapshots, and lottery mechanics, is handled off-chain by the protocol backend.

The contract supports the core staking vault flow by allowing users to lock EVT tokens into administrator-defined pools and withdraw them once the relevant lock period has expired. It maintains per-stake records, per-user balances per pool, per-pool totals, and the overall amount locked in the vault. Administrative functionality includes creation and configuration of pools through lock-duration settings, while emergency controls allow authorized roles to pause all lock and withdrawal activity and, when necessary, recover ERC-20 tokens or native BNB from the contract during incident handling. This keeps the on-chain implementation focused on token custody, lock enforcement, accounting, and essential administrative and emergency operations.

### Privileged roles

- `DEFAULT_ADMIN_ROLE`: Manages granting or revoking privileged roles.
- `ADMIN_ROLE`: Manages core administrative actions, including configuring pool lock durations.
- `EMERGENCY_ROLE`: Handles incident-response actions, including pausing or unpausing the contract and executing emergency recovery of ERC-20 tokens or force-sent BNB while paused.
- `UPGRADE_ROLE`: Authorizes contract implementation upgrades through the UUPS proxy mechanism.

## Potential Risks

- **Single Points of Failure and Control:** The project is fully or partially centralized, introducing single points of failure and control. This centralization can lead to vulnerabilities in decision-making and operational processes, making the system more susceptible to targeted attacks or manipulation.
- **Flexibility and Risk in Contract Upgrades:** The project's contracts are upgradable, allowing the administrator to update the contract logic at any time. While this provides flexibility in addressing issues and evolving the project, it also introduces risks if upgrade processes are not properly managed or secured, potentially allowing for unauthorized changes that could compromise the project's integrity and security.
- **Absence of Upgrade Window Constraints:** The contract suite allows for immediate upgrades without a mandatory review or waiting period, increasing the risk of rapid deployment of malicious or flawed code, potentially compromising the system's integrity and user assets.
- **Emergency Withdrawal Can Break Accounting Consistency:** The `emergencyWithdraw` function intentionally allows privileged operators to recover ERC-20 tokens from the contract without updating internal accounting variables such as per-stake, per-pool, or total locked balances. While this behavior is designed for emergency response and is only available when the contract is paused, it creates a scenario where on-chain accounting may no longer accurately reflect the actual token balance held by the vault. As a result, following emergency intervention, recorded locked amounts may become inconsistent with recoverable assets unless additional off-chain reconciliation or operational remediation is performed.
- **Heavy Reliance on Off-Chain Backend Logic:** A significant portion of the protocol's functional and economic logic is enforced exclusively by the off-chain Go backend rather than by the smart contract itself. This includes reward calculations, EVT-days accumulation, contribution scoring, lottery handling, soft-to-hard pool transitions, and per-pool CAP tracking. While this design keeps the on-chain contract minimal, it also introduces operational and trust dependencies on the correctness, security, and availability of the backend systems. If the backend is misconfigured, unavailable, or compromised, important protocol rules may be applied incorrectly or not enforced at all, even though the smart contract itself continues to operate as designed.
- **Admin Lock Duration Update Can Front-Run User Lock:** The `lock()` function uses the current `poolLockDuration[poolId]` value at execution time to determine the stake expiry. Because `setPoolLockDuration()` can update this value at any time through the `DEFAULT_ADMIN_ROLE`, a user's pending `lock()` transaction may be front-run by an admin duration update. As a result, a user who submitted a transaction expecting the previous lock period may instead have their tokens locked for a newly increased duration once the transaction is mined. This creates timing dependence on privileged admin actions and may lead to users being locked under terms they did not anticipate at submission time.
- **String-Based Pool IDs Increase Gas Costs:** The contract uses `string` values for `poolId`, which introduces additional gas overhead compared to a fixed-size type such as `bytes32`. Dynamic string handling is generally more expensive, particularly for storage writes, comparisons, and hashing. Additionally, `poolId` currently has no explicit length restriction, meaning longer values may increase execution costs unnecessarily. While this may be manageable in a system with a small, backend-controlled set of pools, it remains a design choice that trades gas efficiency for readability and flexibility.

# Findings

## Vulnerability Details

### F-2026-15337 - Floating Pragma - Info

**Description:** In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.20`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The `SyFuStaking` contract currently use floating pragma declarations `^0.8.28`. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

**Assets:**

- `SyFuStaking.sol` [[https://github.com/SyFu-Project/syfu\\_staking](https://github.com/SyFu-Project/syfu_staking)]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

### Recommendations





**Remediation:**

It is recommended to lock the pragma version to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates. For example, instead of using `^0.8.xx`, explicitly define the version with `pragma solidity 0.8.xx;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: [Solidity GitHub releases](#) or [Solidity Bugs by Version](#). Choose a compiler version with a good track record for stability and security.

**Resolution:**

Fixed in `84f482e`, the pragma version was locked at `0.8.28`.

## F-2026-15341 - Lack of Event Emitting for Emergency Withdraw Operations - Info

### Description:

The **SyFuStaking** contract includes emergency recovery functions that allow accounts with **EMERGENCY\_ROLE** to withdraw ERC-20 tokens or native BNB from the contract when it is paused. This mechanism is intended for emergency fund recovery and administrative intervention.

However, the corresponding functions do not emit an event upon successful execution:

```
function emergencyWithdraw(
    IERC20 rescueToken,
    address to,
    uint256 amount
) external onlyRole(EMERGENCY_ROLE) whenPaused {
    if (to == address(0)) revert ZeroAddress();
    rescueToken.safeTransfer(to, amount);
}

function rescueBNB(address payable to, uint256 amount)
    external onlyRole(EMERGENCY_ROLE) whenPaused
{
    if (to == address(0)) revert ZeroAddress();
    (bool ok, ) = to.call{value: amount}("");
    if (!ok) revert NativeTransferFailed();
}
```

The absence of an event makes it difficult to track admin-specific actions off-chain, which reduces transparency and complicates monitoring and auditing. It is recommended to emit events for this operation to ensure proper visibility, support off-chain indexing, and facilitate debugging, especially for critical administrative and user-facing fund transfers.

### Assets:

- SyFuStaking.sol [[https://github.com/SyFu-Project/syfu\\_staking](https://github.com/SyFu-Project/syfu_staking)]

### Status:

Fixed

## Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

---

## Recommendations

**Remediation:** Consider emitting events within the mentioned affected functions to enhance transparency and ensure reliable off-chain tracking of key state changes. Emitting events not only improves monitoring of critical storage variable updates but also supports indexing services, facilitates auditing, and provides better visibility into administrative actions.

**Resolution:** Fixed in `4f1dd60`, `EmergencyWithdrawn` and `BNBRescued` events were introduced and emitted in the appropriate emergency functions:

```
function emergencyWithdraw(
    IERC20 rescueToken,
    address to,
    uint256 amount
) external onlyRole(EMERGENCY_ROLE) whenPaused {
    ...
    emit EmergencyWithdrawn(rescueToken, to, amount);
}

function rescueBNB(address payable to, uint256 amount)
    external onlyRole(EMERGENCY_ROLE) whenPaused
{
    ...
    emit BNBRescued(to, amount);
}
```

## F-2026-15342 - Redundant Imports - Info

### Description:

The **SyFuStaking** contract explicitly imports and directly inherits **Initializable** contract:

```
import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";

contract SyFuStaking is
    Initializable, //<--
    UUPSUpgradeable,
    AccessControlUpgradeable,
    PausableUpgradeable,
    ReentrancyGuardUpgradeable
{
    ...
}
```

However, **UUPSUpgradeable** already inherits from **Initializable**, so the direct inheritance is redundant. Since **Initializable** is already reachable through **UUPSUpgradeable**, explicitly inheriting it again is unnecessary and slightly increases code verbosity without adding functionality.

Additionally, the contract imports and uses **IERC20** alongside **SafeERC20**:

```
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

...

using SafeERC20 for IERC20;
```

**SafeERC20** already relies on **IERC20**, so the explicit **IERC20** import in **SyFuStaking** appears redundant from a code-maintenance perspective. Keeping both imports increases verbosity and can make the dependency surface look larger than it actually is.

This may lead to reduced code clarity and maintainability, as redundant imports and inheritance can make the contract appear more complex than necessary and may create minor confusion for reviewers or future maintainers.

### Assets:

- SyFuStaking.sol [[https://github.com/SyFu-Project/syfu\\_staking](https://github.com/SyFu-Project/syfu_staking)]

### Status:

Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

---

## Recommendations

**Remediation:** Remove redundant direct inheritance (**Initializable**) and unnecessary imports (**Initializable**, **IERC20**) to simplify the codebase and improve readability.

**Resolution:** Fixed in [8360642](#), redundant direct inheritance of **Initializable** and unnecessary imports of **Initializable**, **IERC20** were removed.

## F-2026-15344 - Inconsistent Granular Role Separation - Info

### Description:

The **SyFuStaking** contract defines dedicated privileged roles for specific administrative functions, such as `EMERGENCY_ROLE` and `UPGRADE_ROLE`. However, some routine administrative actions still rely on `DEFAULT_ADMIN_ROLE`, for example:

```
function setPoolLockDuration(string calldata poolId, uint256 duration)
    external onlyRole(DEFAULT_ADMIN_ROLE)
{
    poolExists[poolId] = true;
    poolLockDuration[poolId] = duration;
    emit PoolLockDurationSet(poolId, duration);
}
```

Using `DEFAULT_ADMIN_ROLE` for day-to-day administrative operations increases the impact of a potential compromise. If the key controlling this role is compromised, an attacker could perform unauthorized configuration changes, disrupt expected protocol behavior, or gain control over critical protocol parameters.

This design results in inconsistent role segregation, where some responsibilities are properly isolated while others remain assigned to an overly privileged role.

### Assets:

- SyFuStaking.sol [[https://github.com/SyFu-Project/syfu\\_staking](https://github.com/SyFu-Project/syfu_staking)]

### Status:

Fixed

### Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

### Recommendations

**Remediation:**

Introduce a dedicated `ADMIN_ROLE` for routine administrative and operational actions like `setPoolLockDuration()`, and reserve `DEFAULT_ADMIN_ROLE` exclusively for role management.

**Resolution:**

Fixed in `8ca0aa5`, a new `ADMIN_ROLE` was introduced for routine administrative and operational actions instead of `DEFAULT_ADMIN_ROLE`:

```
function setPoolLockDuration(string calldata poolId, uint256 duration)
    external onlyRole(ADMIN_ROLE)
{...}
```

## Disclaimers

### Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

### Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.



## Appendix 1. Definitions

### Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hackenio/severity-formula)

| Severity | Description                                                                                                                                                                                                                  |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Critical | Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.                                                                                       |
| High     | High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.                                |
| Medium   | Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category. |
| Low      | Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.                                                                                           |

### Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

## Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

| Scope Details          |                                                                                                         |
|------------------------|---------------------------------------------------------------------------------------------------------|
| Repository             | <a href="https://github.com/SyFu-Project/syfu_staking">https://github.com/SyFu-Project/syfu_staking</a> |
| Initial Commit         | ed757003bb95409642b20fc754d7a25285d4912f                                                                |
| Final Commit           | 8ca0aa51e881664399c7b8b42606a375c7a3c4b2                                                                |
| Whitepaper             | N/A                                                                                                     |
| Requirements           | <a href="https://github.com/SyFu-Project/syfu_staking">https://github.com/SyFu-Project/syfu_staking</a> |
| Technical Requirements | <a href="https://github.com/SyFu-Project/syfu_staking">https://github.com/SyFu-Project/syfu_staking</a> |

| Asset                                                                                                                       | Type           |
|-----------------------------------------------------------------------------------------------------------------------------|----------------|
| SyFuStaking.sol [ <a href="https://github.com/SyFu-Project/syfu_staking">https://github.com/SyFu-Project/syfu_staking</a> ] | Smart Contract |

## Appendix 3. Additional Valuables

### Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

### Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.